

ECE 457B – Assignment 1

Will McIntyre (21009235)

September 25th, 2026

Q1.1

To prove that the function $f(\theta) = \left||y - X\theta|\right|_2^2$ is convex, we must show that its Hessian matrix is positive semi-definite.

First, expand the function into matrix notation:

$$f(\theta) = (y - X\theta)^T(y - X\theta)$$
$$= (y^T - \theta^T X^T)(y - X\theta)$$
$$= y^T y - y^T X\theta - \theta^T X^T y + \theta^T X^T X\theta$$

The two middle terms are equal and scalar:

$$f(\theta) = y^T y - 2y^T X\theta + \theta^T X^T X\theta$$

Next, calculate the gradient, recognizing that $2y^T X\theta$ is a linear term and $\theta^T X^T X\theta$ is a quadratic form with $X^T X$ being symmetric:

$$\nabla f = \frac{\partial f}{\partial \theta} = 0 - 2X^T y + 2X^T X\theta$$

Using the gradient and the fact that $2X^T X\theta$ is a linear term, we can calculate the Hessian:

$$H_f = \frac{\partial^2 f}{\partial \theta^2} = 2X^T A$$

A matrix H is positive semi-definite if $v^T Hv \geq 0$ for any vector $v \in R^{nx1}$:

$$v^T Hv = v^T(2X^T X)v$$
$$= 2(Xv)^T(Xv)$$
$$= 2||Xv||_2^2$$

Because the norm of a vector is always greater than or equal to zero:

$$2\left||Xv|\right|_2^2 \geq 0$$

And thus, the function's Hessian matrix is positive semi-definite and is convex.

Q1.2

First, construct the Lagrangian using the objective function and constraint:

$$\mathcal{L}\{\theta, \lambda\} = f(\theta) + \lambda(g(\theta) - c)$$

Where:

$$f(\theta) = y^T y - 2y^T X\theta + \theta^T X^T X\theta$$
$$g(\theta) = w^T\theta$$
$$c = b$$

So:

$$\mathcal{L}\{\theta, \lambda\} = y^T y - 2y^T X\theta + \theta^T X^T X\theta + \lambda(w^T\theta - b)$$

To find the optimum, we take the gradient of the Lagragian and set it to zero. Because $f(\theta)$ is convex, any critical points are automatically global minimums:

$$\frac{\partial \mathcal{L}}{\partial \theta} = -2X^T y + 2X^T X\theta + \lambda w = 0 \quad (1)$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = w^T\theta - b = 0 \quad (2)$$

Isolating for θ in (1) and knowing that $X^T T = I_n$:

$$2X^T X\theta = 2X^T y - \lambda w$$
$$\theta = X^T y - \frac{\lambda}{2} w \quad (3)$$

Then, sub into (2) and isolate for λ:

$$w^T X^T y - \frac{\lambda}{2} w^T w = b$$
$$\frac{\lambda}{2}\omega^T\omega = \omega^T X^T y - b$$
$$\lambda = \frac{2(\omega^T X^T y - b)}{\omega^T\omega} \quad (4)$$

For the final answer, sub (4) into (3):

$$\min_\theta \left|\left|y - X\theta\right|\right|_2^2 = \theta = X^T y - \frac{(\omega^T X^T y - b)}{\omega^T\omega} w = X^T y - \left(\frac{\omega^T X^T y - b}{\|\omega\|_2^2}\right)\omega$$

Q2.1

Find the expected value of the random variable Z:

$$\mathbb{E}[Z] = \mathbb{E}[|X - Y|^2] = \mathbb{E}[X^2 - 2XY - Y^2]$$

Due to linearity of expectation and that X and Y are independent variables:

$$\mathbb{E}[Z] = \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[Y] - \mathbb{E}[Y^2] \tag{5}$$

Now, find the expectation of the random variables X and Y sampled from the interval [0,1]. The probability density function over this interval, to ensure probability adds to zero, is $f(x) = 1$.

$$\mathbb{E}[Y] = \mathbb{E}[X] = \int_0^1 xf(x)dx = \int_0^1 xdx = \frac{1}{2}$$
$$\mathbb{E}[Y^2] = \mathbb{E}[X^2] = \int_0^1 x^2 f(x)dx = \int_0^1 x^2 dx = \frac{1}{3}$$

Subbing into (5) finds the expected value of Z:

$$\mathbb{E}[Z] = \frac{1}{3} - 2\left(\frac{1}{2}\right)\left(\frac{1}{2}\right) - \frac{1}{3} = \frac{1}{6}$$

To find the variance of Z:

$$Var(Z) = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 \tag{6}$$

Find $\mathbb{E}[Z^2]$:

$$\mathbb{E}[Z^2] = \mathbb{E}[(X - Y)^4] = \mathbb{E}[X^4 - 4X^3Y + 6X^2Y^2 - 4XY^3 + Y^4]$$

Once again, due to linearity of expectation and because X and Y are independent variables:

$$\mathbb{E}[Z^2] = \mathbb{E}[X^4] - 4\mathbb{E}[X^3]\mathbb{E}[Y] + 6\mathbb{E}[X^2]\mathbb{E}[Y^2] - 4\mathbb{E}[X]\mathbb{E}[Y^3] + \mathbb{E}[Y^4] \tag{7}$$

$$\mathbb{E}[Y^3] = \mathbb{E}[X^3] = \int_0^1 x^3 f(x)dx = \int_0^1 x^3 dx = \frac{1}{4}$$
$$\mathbb{E}[Y^4] = \mathbb{E}[X^4] = \int_0^1 x^4 f(x)dx = \int_0^1 x^4 dx = \frac{1}{5}$$
$$\mathbb{E}[Z^2] = \frac{1}{5} - 4\left(\frac{1}{4}\right)\left(\frac{1}{2}\right) + 6\left(\frac{1}{3}\right)\left(\frac{1}{3}\right) - 4\left(\frac{1}{2}\right)\left(\frac{1}{4}\right) + \frac{1}{5} = \frac{1}{15}$$

Subbing the result for $\mathbb{E}[Z^2]$ and $E[Z]^2$ into (6):

$$Var(Z) = \frac{1}{15} - \left(\frac{1}{6}\right)^2 = \frac{7}{180}$$

Q2.2

A python script plots the average and standard deviation of both Manhattan and Squared Euclidean distances for 100 independent and uniform points from the unit cube:

```
import numpy as np
import matplotlib.pyplot as plt

dimensions = [2**i for i in range(11)] # [1, 2, 4, 8, ... 1024]
num_points = 100

# Initialize lists to store the statistics for both metrics
sq_eucl_means = []
sq_eucl_stds = []
manhattan_means = []
manhattan_stds = []

# get the upper triangle indices for the matrix (all unique combos)
i, j = np.triu_indices(num_points, k=1)

#
rng = np.random.default_rng()

for d in dimensions:
    # make samples of 100 points from a d dimensional unit cube (random)
    points = rng.uniform(0, 1, size=(num_points, d))

    # Calculate dimensional differences for all 4,950 unique pairs simultaneously
    diffs = points[i] - points[j]

    # Calculate Squared Euclidean Distances (sum of squared differences)
    sq_eucl_dists = np.sum(diffs ** 2, axis=1)
    sq_eucl_means.append(np.mean(sq_eucl_dists))
    sq_eucl_stds.append(np.std(sq_eucl_dists))

    # Calculate Manhattan Distances (sum of absolute differences)
    manhattan_dists = np.sum(np.abs(diffs), axis=1)
    manhattan_means.append(np.mean(manhattan_dists))
    manhattan_stds.append(np.std(manhattan_dists))

# Generate the plots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Mean of Manhattan and Squared Euclidean Distances
ax1.plot(dimensions, manhattan_means, label='Manhattan', marker='o', color='red')
```

```
ax1.plot(dimensions, sq_eucl_means, label='Squared Euclidean', marker='o',
color='blue')
ax1.set_xscale('log', base=2)
ax1.set_title('Mean Distance vs. Dimension (d)')
ax1.set_xlabel('Dimension (d)')
ax1.set_ylabel('Value')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.7)

# Plot 2: Standard Deviation of Manhattan and Squared Euclidean Distances
ax2.plot(dimensions, manhattan_stds, label='Manhattan', marker='s', color='red')
ax2.plot(dimensions, sq_eucl_stds, label='Squared Euclidean', marker='s',
color='blue')
ax2.set_xscale('log', base=2)
ax2.set_title('Standard Deviation vs. Dimension (d)')
ax2.set_xlabel('Dimension (d)')
ax2.set_ylabel('Value')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

The code generates random points at each dimension and calculates the Manhattan and Squared Euclidean distances for each possible combination (without repeats). It then calculates the mean distance for each dimension and the standard deviation, plotted in Figure 1 and Figure 2.

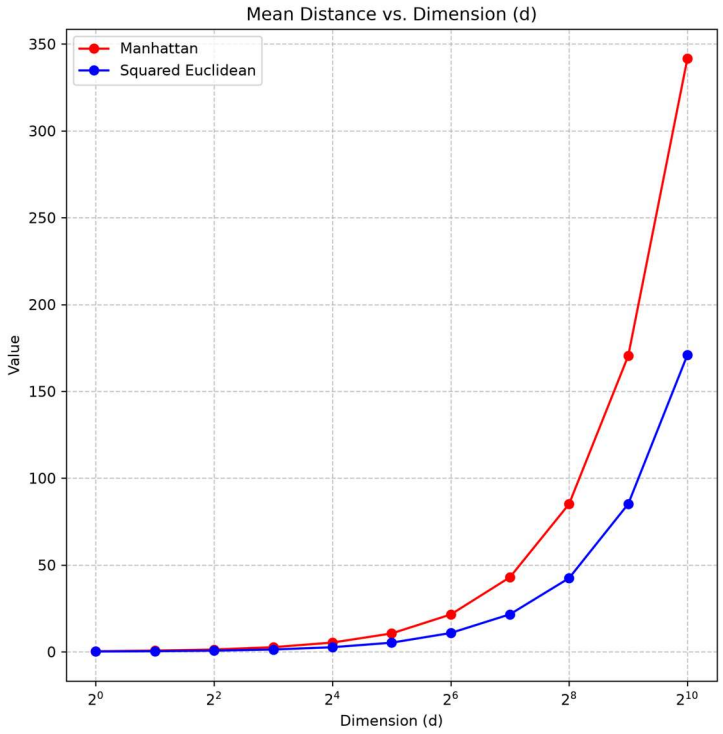


Figure 1: Mean Distance vs Dimension for Manhattan and Squared Euclidean Distances

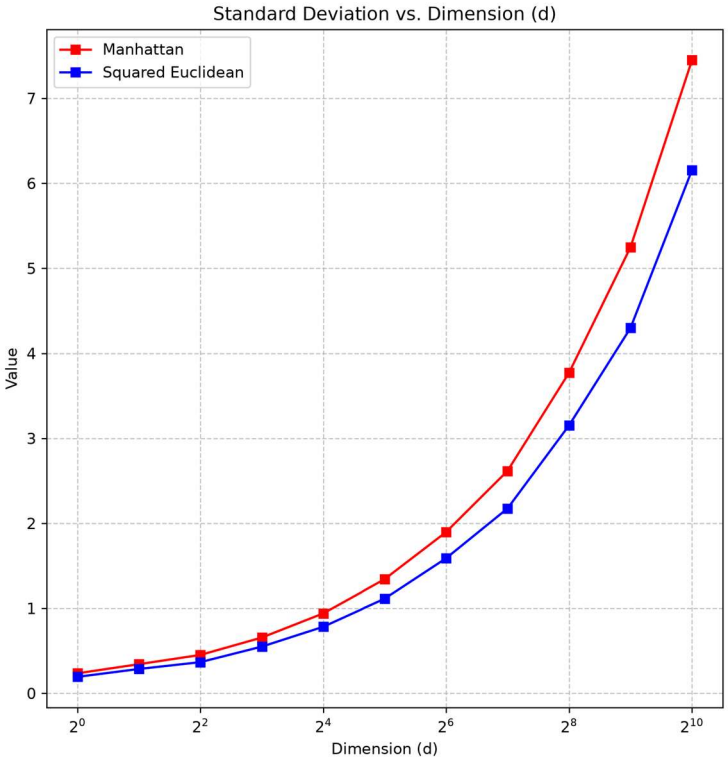


Figure 2: Standard Deviation vs Dimension for Manhattan and Squared Euclidean Distances

These figures display a few conclusions. First, the mean distances for both metrics scale linearly with the number of dimensions. The exponential on the log scale indicates a direct proportional relationship to the dimension size. Also, mean Manhattan distance is twice the distance of the Squared Euclidean distance for each dimension. Additionally, Manhattan distances consistently have higher standard deviation across all dimensions in the experiment, meaning that it has higher variance. Overall, these graphs show the curse of dimensionality; as we go into higher-dimension space, the mean distance increases at a rapid rate and the standard deviation becomes mathematically negligible relative to those distances. This means that every randomly sampled point ends up being almost the same distance away from every other point at higher dimensions.

Q2.3

Using linearity of expectation:

$$E[R] = E[Z_1 + Z_2 + \cdots + Z_d] = E[Z_1] + E[Z_2] + \cdots + E[Z_d]$$

Because the points are sampled uniformly from a d-dimensional cube, every dimension $Z_i$is identically distributed and shares the same 1D expected value of 1/6, from Part 1. Thus:

$$E[R] = dE[Z_i] = \frac{d}{6}$$

For variance, we know that points X and Y are sampled independently and therefore can use the Additivity of Variance:

$$Var[R] = Var[Z_1 + Z_2 + \cdots + Z_d] = Var[Z_1] + Var[Z_2] + Var[Z_d]$$

Like the expected value, every dimension Z_i is identically distributed thus shares the same variance as that calculated in Part 1:

$$Var[R] = dVar[Z_d] = \frac{7d}{180}$$

Q2.4

To prove that at high dimensions, most points become approximately the same distance, we begin with the derivation shown in the assignment, which shows the maximum probability that a random variable will deviate from its expected average by amount a:

$$P(|Z - E[Z]| \geq a) \leq \frac{Var[Z]}{a^2}$$

To map this equation to random variables in R^d, use the definition of $R = Z_1 + Z_2 + \cdots + Z_d$:

$$P(|R - E[R]| \geq a) \leq \frac{Var[R]}{a^2}$$

Define a as a relative percentage of the expected value, $a = cE[R]$ where $c > 0$:

$$P(|R - E[R]| \geq cE[R]) \leq \frac{Var[R]}{cE[R]^2}$$

Plug in the calculated variance and expected value of our specific distance variable R:

$$P\left(|R - \mathbb{E}[R]| \geq c\frac{d}{6}\right) \leq \frac{\frac{7d}{180}}{\left(c\frac{d}{6}\right)^2} = \frac{7}{5c^2d}$$

If we then take the limit as $d \to \infty$:

$$\lim_{d\to\infty} \frac{7}{5c^2d} = 0$$

Therefore, as dimensions increase, the probability that R is far away, relative to its expected value, goes to zero.

Q2.5

It is shown in the assignment that l, the edge length of the smallest hyper-cube in d-dimensional space that holds all k nearest neighbours to the test point takes on the following value:

$$l^d \approx \frac{k}{n}$$

$$l = \left(\frac{k}{n}\right)^{\frac{1}{d}}$$

For k = 10 and $n = 1000$, the approximate values of l for varying $d = 10^{\{0,1,2,3,4\}}$ are shown in Table 1.

Table 1: Approximated Edge Length Across Dimensions

Dimension	Approximate l Value
1	0.01
10	0.6309
100	0.9549
1000	0.9954
10000	0.9995

This table shows that as the dimension increases, the edge length rapidly approaches 1. Because the entire dataset is contained within a unit cube of length 1, an edge length approaching 1 means that the bounding box for k-nearest neighbours must stretch across almost the entire space in every direction. In kNN, this means that neighbours are no longer truly local. To get just 10 nearest samples in a dataset of 1000 samples, you'd look at almost the whole cube, and those points classified as near are located at extreme edges.

Q2.6

To maintain genuine locality in higher dimensions, you would need more samples. To maintain an l value of 0.1 given k = 10 in only 10 dimensions:

$$n = \frac{k}{l_d} = \frac{10}{0.1^{\frac{1}{10}}} = 10^{11}\ samples$$

This shows that increasing the number of samples for higher-dimensional datasets with more features is unsustainable, as achieving the required number of samples can become unrealistic, and even impossible at higher dimensions.

Q3.1

The code below implements a KNN using the Euclidean Distance and Cosine Similarity in a $myKNN$ class. The class takes in the number of neighbours k and metric = {“Euclidian”, “Cosine”} on construction. It also can accept training and validation sets, and then the function run_evaluation() takes in all the data, the k value and metric, evaluates the performance and outputs the training, validation and test error and accuracy. Note that the question asks for performance on the validation set and test set, but I added training accuracy for completion and later questions.

```
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

def euclidean_distance(vector_1, vector_2):
    difference = vector_1 - vector_2
    return np.sum(difference**2)**(1/2)

def cosine_similarity(vector_1, vector_2):
    dot_product = np.dot(vector_1, vector_2)
    norm_1 = np.linalg.norm(vector_1)
    norm_2 = np.linalg.norm(vector_2)

    if norm_1 == 0 or norm_2 == 0:
        return 0.0

    cos_theta = dot_product/(norm_1*norm_2)
    return cos_theta

class myKNN:
    def __init__(self, k, metric='euclidian'):
        self.k = k
        self.metric = metric

    def set_validation_data(self, X_validation, Y_validation):
        self.X_validation = np.array(X_validation)
        self.Y_validation = np.array(Y_validation)

    def set_train_data(self, X_train, Y_train):
        # store the training data in our case:
        # (glucose concentration, BMI, age)
        self.X_train = np.array(X_train)
        # note this is the target label
```

```
        self.Y_train = np.array(Y_train)

    def predict(self, X):
        # predict a target for each point in the set X using
        # the training data
        predictions = [self._predict(x) for x in np.array(X)]
        return np.array(predictions)

    def _predict(self, x):
        # predict the expected target using knn for the point x

        # calculate distances based on test point x for all training points
        if self.metric == "Euclidean":
            distances = [euclidean_distance(v, x) for v in self.X_train]
            # get the k lowest distances
            k_indices = np.argsort(distances)[:self.k]
        elif self.metric == "Cosine":
            distances = [cosine_similarity(v, x) for v in self.X_train]
            # cosine is similarity metric: higher is better, take last elements
            # and reverse them
            k_indices = np.argsort(distances)[-self.k:][::-1]
        else:
            raise ValueError("Unsupported metric. " \
            "Can only use 'Euclidean' or 'Cosine'")

        # get the most common labels for the k nearest neighbours:
        labels = self.Y_train[k_indices]
        counter = Counter(labels)
        most_common_target = counter.most_common(1)[0][0]

        return most_common_target

    def evaluate_performance(self, X_test, Y_test):
        # evaluation function to measure accuracy for validation/test sets
        training_predictions = self.predict(self.X_train)
        training_accuracy = np.mean(training_predictions == self.Y_train)
        training_error = 1 - training_accuracy

        validation_predictions = self.predict(self.X_validation)
        validation_accuracy = np.mean(validation_predictions ==
                                      self.Y_validation)
        validation_error = 1 - validation_accuracy

        test_predictions = self.predict(X_test)
        test_accuracy = np.mean(test_predictions == Y_test)
```

```
        test_error = 1 - test_accuracy

        return (training_accuracy, training_error, validation_accuracy,
                validation_error, test_accuracy, test_error)

def run_evaluation(X_train, Y_train, X_val, Y_val, X_test, Y_test, k, metric):
    knn = myKNN(k = k, metric=metric)
    knn.set_train_data(X_train, Y_train)
    knn.set_validation_data(X_val, Y_val)

    train_acc, train_err, val_acc, val_err, test_acc, test_err = \
        knn.evaluate_performance(X_test, Y_test)
    print(f"Custom Implementation Metrics for k={k}, distance={metric}:")
    print(f"Training Accuracy: {train_acc:.4f} (Error: {train_err:.4f})")
    print(f"Validation Accuracy: {val_acc:.4f} (Error: {val_err:.4f})")
    print(f"Test Accuracy: {test_acc:.4f} (Error: {test_err:.4f})\n")
```

Q3.2

To split the dataset into the desired training ratios, I used sklearn's train_test_split() function to split the dataset as desired.

```
def question_3_2(X, Y):
    # The splits required by the assignment
    split_configs = {
        "80/10/10": (0.80, 0.10, 0.10),
        "34/33/33": (0.34, 0.33, 0.33),
        "25/25/50": (0.25, 0.25, 0.50)
    }
    metrics = ["Euclidean", "Cosine"]
    k = 1 # 1 nearest neighbour for this section

    for split_name, (train_r, val_r, test_r) in split_configs.items():
        print(f"\n{'='*30}")
        print(f"=== Data Split: {split_name} ===")
        print(f"{'='*30}")

        # Split off the test set first
        X_temp, X_test, Y_temp, Y_test = train_test_split(
            X, Y, test_size=test_r, shuffle=False
        )

        # Split the remaining data into train and validation
        val_ratio_adjusted = val_r / (train_r + val_r)
        X_train, X_val, Y_train, Y_val = train_test_split(
            X_temp, Y_temp, test_size=val_ratio_adjusted, shuffle=False
        )

        for metric in metrics:
            run_evaluation(X_train, Y_train, X_val, Y_val, X_test, Y_test, k,
metric)

            # predict and find accuracy using SkLearn
            sk_knn = KNeighborsClassifier(n_neighbors=k, metric=metric.lower())
            sk_knn.fit(X_train, Y_train)

            sk_train_acc = accuracy_score(Y_train, sk_knn.predict(X_train))
            sk_train_err = 1 - sk_train_acc
            sk_val_acc = accuracy_score(Y_val, sk_knn.predict(X_val))
            sk_val_err = 1 - sk_val_acc
            sk_test_acc = accuracy_score(Y_test, sk_knn.predict(X_test))
            sk_test_err = 1 - sk_test_acc
            print(f"SkLearn Metrics for k={k}, distance={metric}:")
```

```
            print(f"Training Accuracy: {sk_train_acc:.4f} (Error:
{sk_train_err:.4f})")
            print(f"Validation Accuracy: {sk_val_acc:.4f} (Error:
{sk_val_err:.4f})")
            print(f"Test Accuracy: {sk_test_acc:.4f} (Error:
{sk_test_err:.4f})\n")
```

The output from this code follows in Table 2:

Table 2: Data Splits and Validation Error Rate Across Different Metrics

Split	Metric	Custom Validation Error Rate (%)	Sklearn Validation Error Rate (%)
80/10/10	Euclidean	36.36	36.36
80/10/10	Cosine	42.86	42.86
34/33/33	Euclidean	35.43	35.43
34/33/33	Cosine	43.31	43.31
25/25/50	Euclidean	35.94	35.94
25/25/50	Cosine	49.48	49.48

Observing these validation error rates across the different splits and metrics for both our custom and sklearn implementation, the 34/33/33 split with Euclidean distance has the lowest validation error rate. Across all splits, Euclidean distance gives lower validation error rates. This makes logical sense given the features, as Glucose, BMI and Age should be related through numerical distance. For example, a patient with Glucose of 100 and BMI of 25 would have a similar vector angle to someone with Glucose of 200 and BMI of 50, but these are two very different health outcomes. In typical machine learning examples, an 80/10/10 split usually performs best but because we are dealing with a relatively small dataset, the 33% validation set happened to map very cleanly to the structure of its 34% training set. As we decreased to the 25/25/50 set, we see the worst results which makes more sense as less of the model is used for training.

Q3.3

Using AI for reference and plotting[1], I generated the following code for comparing the metrics, splitting of data and additionally k values using my custom kNN implementation:

```
def question_3_3(X, Y):
    # Ensure splits match your final Q3.2 table
    split_configs = {
        "80/10/10": (0.80, 0.10, 0.10),
        "34/33/33": (0.34, 0.33, 0.33),
        "25/25/50": (0.25, 0.25, 0.50)
    }
    metrics = ["Euclidean", "Cosine"]
    k_values = [1, 3, 5, 11]

    # Set up a 3x2 grid for plotting (3 splits, 2 metrics)
    fig, axes = plt.subplots(3, 2, figsize=(12, 14))
    fig.subplots_adjust(hspace=0.4, wspace=0.3)

    # Store the best configuration found
    best_error = float('inf')
    best_config = {}

    for i, (split_name, (train_r, val_r, test_r)) in
enumerate(split_configs.items()):
        # Split the data using a fixed seed so the graphs are reproducible
        X_temp, X_test, Y_temp, Y_test = train_test_split(
            X, Y, test_size=test_r, shuffle=False
        )
        val_ratio_adjusted = val_r / (train_r + val_r)
        X_train, X_val, Y_train, Y_val = train_test_split(
            X_temp, Y_temp, test_size=val_ratio_adjusted, shuffle=False
        )

        for j, metric in enumerate(metrics):
            train_errors = []
            val_errors = []
            test_errors = []

            for k in k_values:
                sk_knn = KNeighborsClassifier(n_neighbors=k,
metric=metric.lower())
                sk_knn.fit(X_train, Y_train)
```

[1] See GenAI Documentation Entry 1

```
                # Calculate Error Rates (1 - Accuracy)
                train_err = 1 - accuracy_score(Y_train, sk_knn.predict(X_train))
                val_err = 1 - accuracy_score(Y_val, sk_knn.predict(X_val))
                test_err = 1 - accuracy_score(Y_test, sk_knn.predict(X_test))

                train_errors.append(train_err)
                val_errors.append(val_err)
                test_errors.append(test_err)

                # Track the best performing model based on validation error
                if val_err < best_error:
                    best_error = val_err
                    best_config = {
                        'k': k,
                        'metric': metric,
                        'split': split_name,
                        'train_err': train_err,
                        'val_err': val_err,
                        'test_err': test_err
                    }

            # Plotting the three error lines for this specific split and metric
            ax = axes[i, j]
            ax.plot(k_values, train_errors, label='Train Error', marker='o',
linestyle='-')
            ax.plot(k_values, val_errors, label='Val Error', marker='s',
linestyle='--')
            ax.plot(k_values, test_errors, label='Test Error', marker='^',
linestyle=':')

            ax.set_title(f"Split: {split_name} | Metric: {metric}")
            ax.set_xlabel("Number of Neighbors (k)")
            ax.set_ylabel("Error Rate")
            ax.set_xticks(k_values)
            ax.legend()
            ax.grid(True, alpha=0.5)

    plt.suptitle("KNN Error Rates Across Splits, Metrics, and k-values",
fontsize=16)
    plt.show()

    print(f"\n{'='*40}")
    print("=== BEST COMBINATION IDENTIFIED ===")
    print(f"{'='*40}")
    print(f"Split:            {best_config['split']}")
```

```
    print(f"Metric:           {best_config['metric']}")
    print(f"k-value:          {best_config['k']}")
    print(f"Training Error:   {best_config['train_err']:.4f}")
    print(f"Validation Error: {best_config['val_err']:.4f}")
    print(f"Test Error:       {best_config['test_err']:.4f}")
```

Figure 3 and Figure 4 display the splits across different number of neighbours for both Euclidean and Cosine metrics along with their train, validation and testing error.

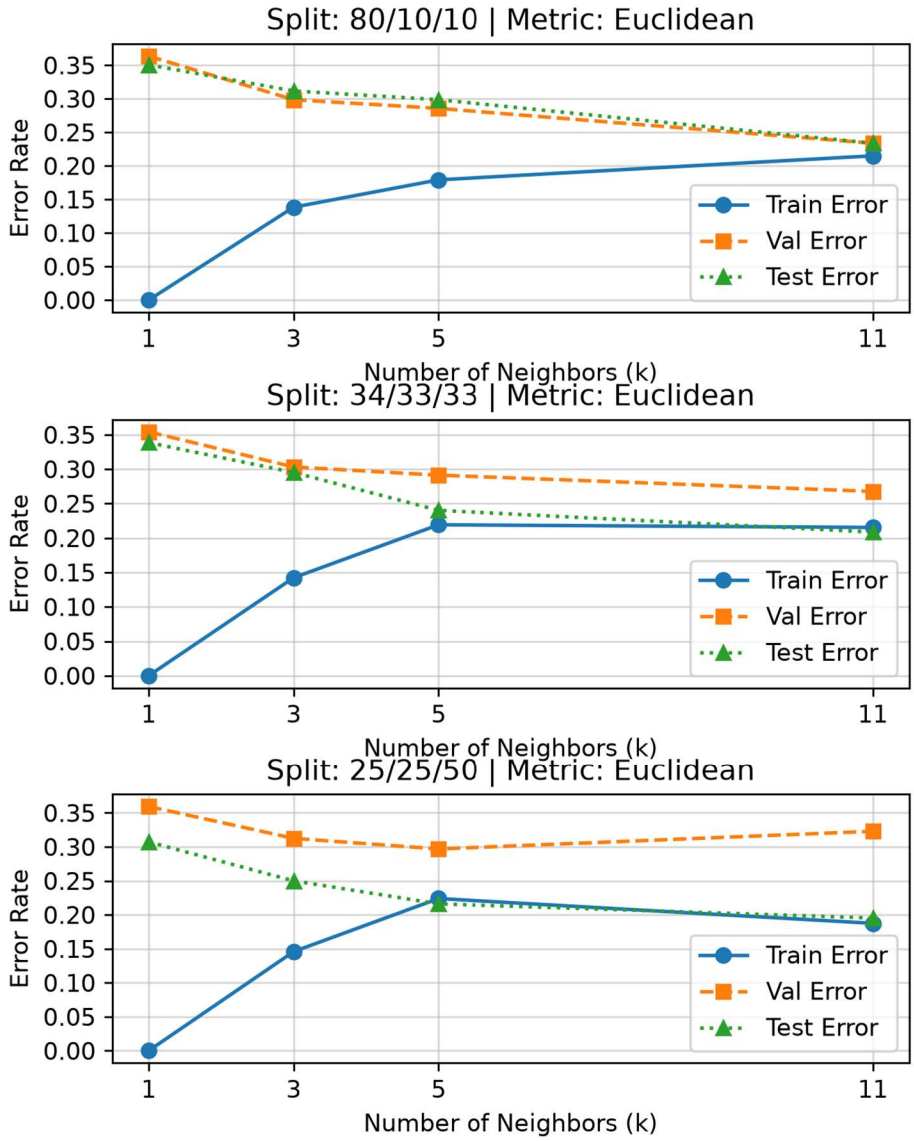


Figure 3: Euclidean Metric Error Across k

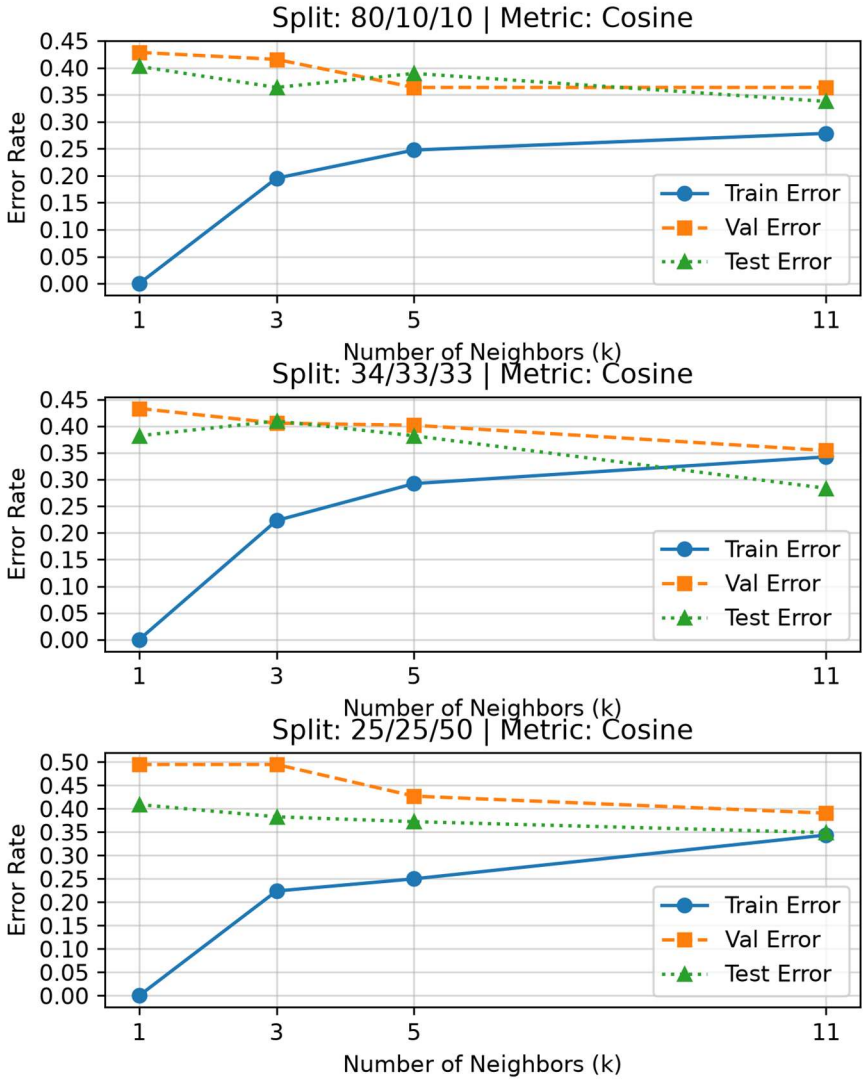


Figure 4: Cosine Metric Error Across k

Based on the graphs generated looking at training, validation and test error, the 80/10/10 data split using the Euclidean metric with k = 11 is the best overall combination for this dataset, with both validation and test error of 23.38%. We choose the model with the lowest validation error, as it is the performance metric used to select the best performing value of hyperparameter, k.

This choice makes sense given the circumstances. At k = 1, we overfit, with zero training error but large validation/test error. k = 11 has a good balance of eliminating pure memorization (lowering variance) of training data while not over smoothing (adding too much bias). Euclidean performance is consistently better than cosine performance, which also makes sense given the nature of the data. As explained in Q3.2, glucose concentration, BMI and age and other health metrics generally relate by numerical distance rather than similarity. For example, if glucose, BMI and age doubled, we would expect very different health results. Finally, an 80/10/10 split is superior because KNN is a

lazy learning algorithm that benefits from having more dense training data compared to the 34% or 25% training splits.

Q4.1

Analysis of the passengers using the dataset's Gender variable yielded the following data on prediction and misclassification:

Table 3: Gender Split of Survivors and Non-Survivors of the Titanic

Group	Survivors	Non-Survivors	Majority Prediction	Errors
Female	233	81	Survived (1)	81
Male	109	468	Did Not Survive (0)	109

To find the misclassified fraction, add both groups of misclassified passengers and divide by the total number of passengers:

$$misclassified\ (\%) = \frac{81 + 109}{233 + 81 + 109 + 478} = \frac{190}{891} = 21.3\%$$

Q4.2

For a categorical variable like gender, there is only one possible way to split the data (like male or female). However, a continuous numerical variable like age has many values, resulting in numerous possible splitting thresholds. An algorithm requires an impurity metric to evaluate these potential thresholds to determine which one creates the least mixed groups. If there was no metric for impurity to determine where exactly to split on numerical variables, then the algorithm would not be effective and could split at a poor numerical point.

Q4.3

Considering a tree with two successive splits along gender and then age yields the following results:

Table 4: Tree Results for Splits Along Gender Followed by Age

Final Group Path	Survivors	Non-Survivors	Majority Prediction	Errors
Female, Age <= 65	233	81	Survived (1)	81
Female, Age > 65	0	0	Empty	0
Male, Age <= 65	108	461	Did Not Survive (0)	108
Male, Age > 65	1	7	Did Not Survive (0)	1

The tree's total error count is 190 passengers, and the error percentage follows:

$$misclassified\ (\%) = \frac{190}{891} = 21.32\%$$

The error rate and count remain the same as Q4.1. If we then reverse the order of the split:

Table 5: Tree Results for Splits Along Age Followed by Gender

Final Group Path	Survivors	Non-Survivors	Majority Prediction	Errors
Age <= 65, Female	233	81	Survived (1)	81
Age <= 65, Male	108	461	Did Not Survive (0)	108
Age > 65, Female	0	0	Empty	0
Age > 65, Male	1	7	Did Not Survive (0)	1

The final groups or total error do not change if we reverse the order. This makes logical sense as leaf nodes represent the intersection (AND) between both rules on its path. Filtering a dataset by Age >=65 AND Female is the same as Female AND Age >=65.

Q4.4

To evaluate the candidate splits, we use the Gini impurity formula for a single node:

$$G(t) = 1 - \sum p_{t,c}^2$$

For binary labels, where p_0 and p_1 are the fractions of each class:

$$G(t) = 1 - (p_0^2 + p_1^2)$$

A python function for calculating the Gini impurity for a single node follows:

```
def calculate_gini(labels):
    if len(labels) == 0:
        return 0.0
    p1 = sum(labels) / len(labels)
    p0 = 1 - p1
    return 1 - (p0**2 + p1**2)
```

We can then calculate the weighted score for the split using:

$$I_{\text{tree}} = \sum \frac{n_t}{N} I(t) = \sum \frac{n_t}{N} G(t)$$

First, for the Gender split:

$$G(t_{\text{female}}) = 1 - \left(\left(\frac{81}{314}\right)^2 + \left(\frac{233}{314}\right)^2\right) = 0.3828$$

$$G(t_{\text{male}}) = 1 - \left(\left(\frac{468}{577}\right)^2 + \left(\frac{109}{577}\right)^2\right) = 0.3064$$

Then, calculate the weighted Gini impurity:

$$I_{tree,gender,gini} = \sum \frac{n_t}{N} G(t) = \frac{314}{891}(0.3828) + \frac{577}{891}(0.3064) = 0.3334$$

For the Age split:

$$G(t_{\leq 65}) = 1 - \left(\left(\frac{542}{883}\right)^2 + \left(\frac{341}{883}\right)^2\right) = 0.4741$$

$$G(t_{>65}) = 1 - \left(\left(\frac{7}{8}\right)^2 + \left(\frac{1}{8}\right)^2\right) = 0.2188$$

Calculating the weighted Gini impurity:

$$I_{tree,age,gini} = \sum \frac{n_t}{N} G(t) = \frac{883}{891}(0.4741) + \frac{8}{891}(0.2188) = 0.4718$$

From this analysis, the gender split gives a lower weighted Gini impurity.

Q4.5

Like Gini impurity, we evaluate the Shannon Entropy for a single node with binary labels:

$$H(t) = -\sum p_{t,c} \log_2 p_{t,c}$$

$$H(t) = -(p_0 \log_2 p_0 + p_1 \log_2 p_1)$$

A function for calculating the Shannon Entropy for a single node follows:

```
def calculate_entropy(labels):
    if len(labels) == 0:
        return 0.0
    p1 = sum(labels) / len(labels)
    p0 = 1 - p1

    entropy = 0.0
    if p1 > 0:
        entropy -= p1 * np.log2(p1)
    if p0 > 0:
        entropy -= p0 * np.log2(p0)
    return entropy
```

Then, the weighted Shannon Entropy is:

$$I_{tree} = \sum \frac{n_t}{N} H(t)$$

For the Gender split:

$$\mathrm{H}(t_{female}) = -\left(\frac{81}{314}\log_2\frac{81}{314} + \frac{233}{314}\log_2\frac{233}{314}\right) = 0.8237$$

$$H(t_{male}) = -\left(\frac{468}{577}\log_2\frac{468}{577} + \frac{109}{577}\log_2\frac{109}{577}\right) = 0.6992$$

The weighted Shannon Entropy:

$$I_{tree,gender,shann} = \frac{314}{891}(0.8237) + \frac{577}{891}(0.6992) = 0.7430$$

For age:

$$H(t_{\leq 65}) = -\left(\frac{542}{883}\log_2\frac{542}{883} + \frac{341}{883}\log_2\frac{341}{883}\right) = 0.9623$$

$$H(> 65) = -\left(\frac{7}{8}\log_2\frac{7}{8} + \frac{1}{8}\log_2\frac{1}{8}\right) = 0.5436$$

And weighted:

$$I_{tree,age,shann} = \left(\frac{883}{891} \times 0.9623\right) + \left(\frac{8}{891} \times 0.5436\right) = 0.9585$$

The gender split yields a weighted entropy of 0.7430, which is lower than the age split that has an entropy of 0.9585. Because lower entropy indicates less mixing and more pure nodes, entropy prefers the same split as Gini impurity. Note that the final submitted code contains the logic for calculating the weighted entropy and impurity.

Q4.6

The default decision tree using Gini impurity with a max depth of 3 and desired random state of 123123 can be found below. The features used to split the decision at depth 2 is Has_Cabin and Pclass.

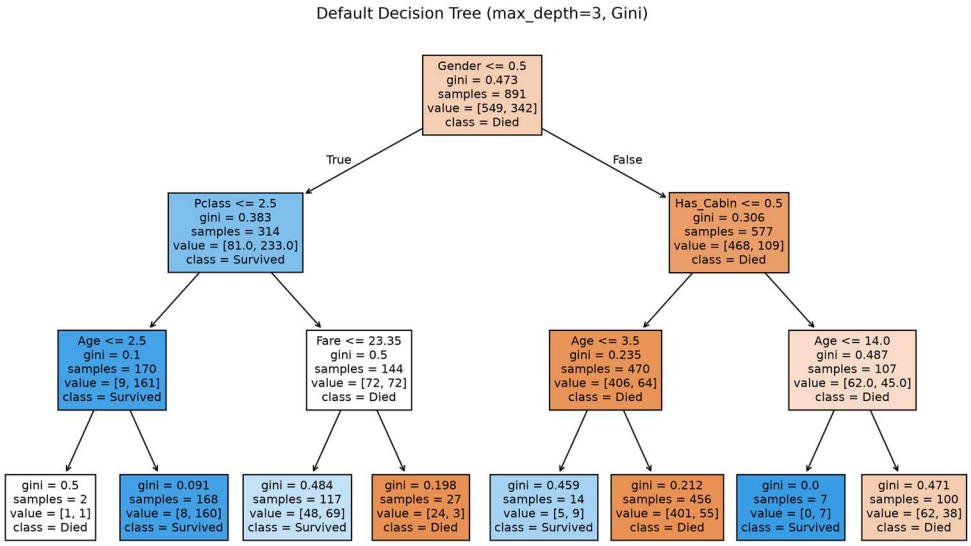


Figure 5: Default Gini Decision Tree

If we set the criterion to “entropy”, seen below, the features stay the same, but the splits and locations change. At depth 3, the splits change from Age <= 2.5, Fare <= 23.35, Age <= 3.5 and Age <= 14.0 to Fare <= 28.856, Fare <= 23.35, Age <= 9.5 and Age <=14.0.

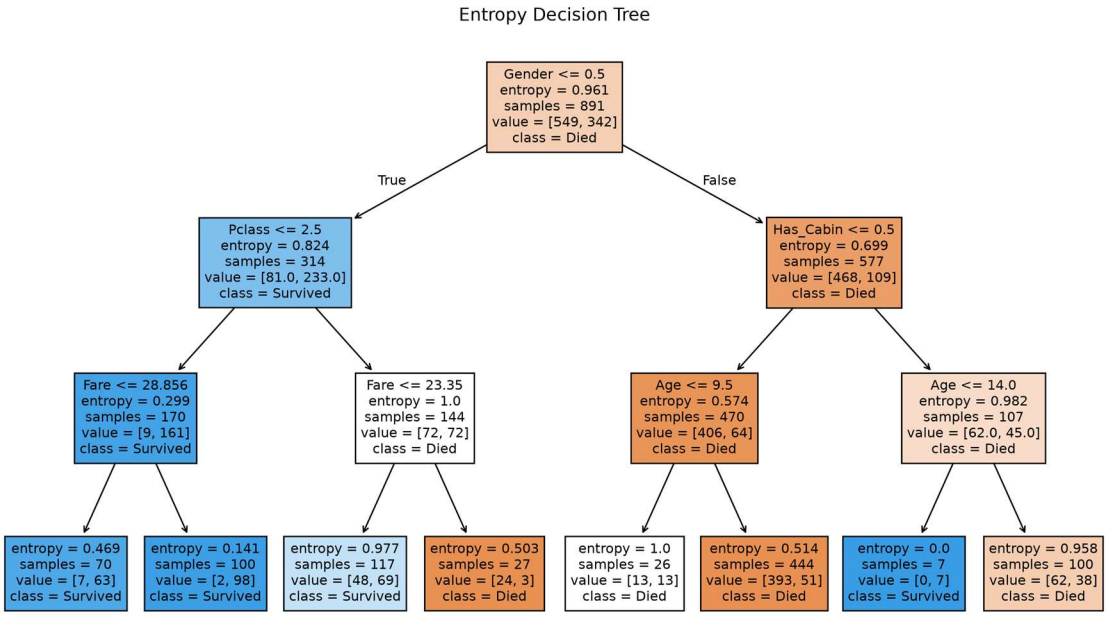


Figure 6: Entropy Criterion Tree

If we change the splitter to "random", it does change the features used at depth 3. Now, the split looks at FamilySize, Embarked and Pclass features.

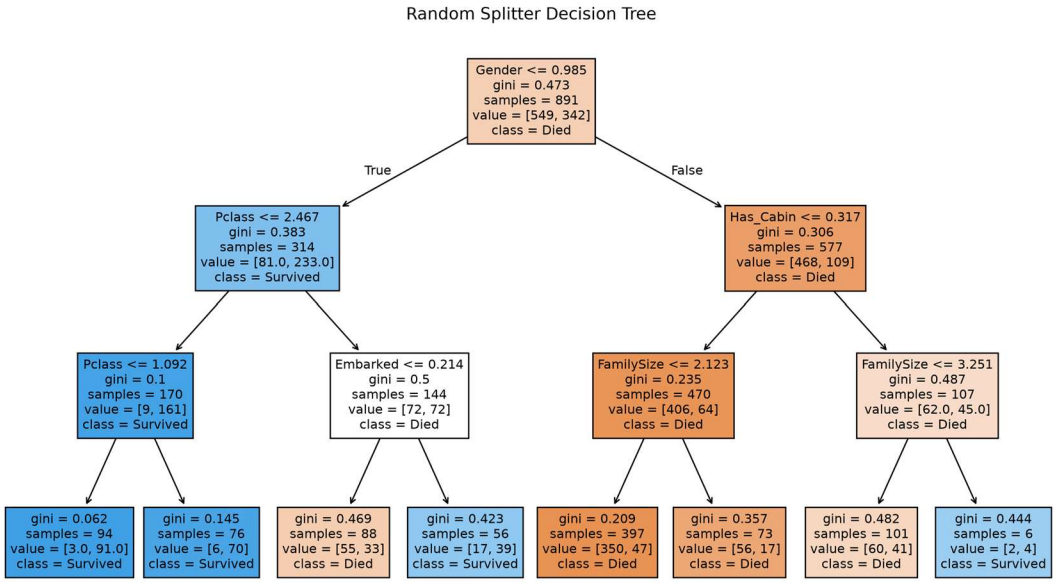


Figure 7: Random Splitter Decision Tree

The code for this can be found below:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

def main():
    # Load the dataset and separate features from the target
    df = pd.read_csv("A1Q4-decision-trees.csv")
    X = df.drop(columns=['Survived'])
    y = df['Survived']

    # default parameters (gini)
    clf_default = DecisionTreeClassifier(max_depth=3, random_state=123123)
    clf_default.fit(X, y)

    plt.figure(figsize=(15, 10))
    plot_tree(clf_default, feature_names=X.columns, class_names=['Died',
'Survived'], filled=True)
    plt.title("Default Decision Tree (max_depth=3, Gini)")
    plt.show()

    # change entropy
```

```
    clf_entropy = DecisionTreeClassifier(max_depth=3, random_state=123123,
criterion="entropy")
    clf_entropy.fit(X, y)

    plt.figure(figsize=(15, 10))
    plot_tree(clf_entropy, feature_names=X.columns, class_names=['Died',
'Survived'], filled=True)
    plt.title("Entropy Decision Tree")
    plt.show()

    # random splitter
    clf_random = DecisionTreeClassifier(max_depth=3, random_state=123123,
splitter="random")
    clf_random.fit(X, y)

    plt.figure(figsize=(15, 10))
    plot_tree(clf_random, feature_names=X.columns, class_names=['Died',
'Survived'], filled=True)
    plt.title("Random Splitter Decision Tree")
    plt.show()

if __name__ == "__main__":
    main()
```

Q4.7

Splitting the dataset into the following train/val/test splits and criterions using scikit-learn decision trees led accuracy readings displayed below:

Table 6: Split and Criterion Accuracy Readings for Decision Tree

Split	Criterion	Training Accuracy (%)	Validation Accuracy (%)	Test Accuracy (%)
80/10/10	Gini	83.85%	78.65%	82.22%
80/10/10	Entropy	83.85%	78.65%	82.22%
34/33/33	Gini	86.42%	78.23%	83.05%
34/33/33	Entropy	86.42%	78.23%	83.05%
25/25/50	Gini	86.04%	82.06%	80.72%
25/25/50	Entropy	85.14%	79.82%	78.92%

Looking at the accuracy scores, there are two impacts to consider. First, the split criterion (Gini impurity vs Entropy). For both the 80/10/10 and 34/33/33 splits, the Gini and Entropy criterion produced the same training, validation and test accuracy. This shows that both models are very similar and often produce the exact same tree structure. The 25/25/50 split diverged as the training set decreased in relative size and the difference between the two formulas can lead to different split conditions.

Second, the splits also change the accuracy readings. Counter-intuitively, the training accuracy readings in the 34% and 25% training datasets are higher than the largest training set (80%). This is likely because the tree is capped at a shallow max depth of 3, and so it is easier for the algorithm to find pure boundaries in this dataset with fewer data points. However, where the 80/10/10 model maximizes training volume, its small test size means its test and validation accuracy can be influenced by variance. The 25/25/50 split using Gini achieved the highest validation accuracy (82.06%), but ultimately the 34/33/33 split yielded the highest test accuracy (83.05%). This incidates that allocating roughly one third of the data to training, validation and testing provides the best balance between giving the model enough data to learn but enough unseen data to prevent memorization.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

def main():
    # Load the dataset
    df = pd.read_csv("A1Q4-decision-trees.csv")
    X = df.drop(columns=['Survived'])
    y = df['Survived']

    split_configs = {
        "80/10/10": (0.80, 0.10, 0.10),
```

```
        "34/33/33": (0.34, 0.33, 0.33),
        "25/25/50": (0.25, 0.25, 0.50)
    }

    results = []

    for split_name, (train_r, val_r, test_r) in split_configs.items():
        # split test set
        X_temp, X_test, y_temp, y_test = train_test_split(
            X, y, test_size=test_r, random_state=123123
        )

        # split into train and validation
        val_ratio_adjusted = val_r / (train_r + val_r)
        X_train, X_val, y_train, y_val = train_test_split(
            X_temp, y_temp, test_size=val_ratio_adjusted, random_state=123123
        )

        # Evaluate both criteria
        for crit in ['gini', 'entropy']:
            clf = DecisionTreeClassifier(max_depth=3, random_state=123123,
criterion=crit)
            clf.fit(X_train, y_train)

            # Calculate accuracy scores
            train_acc = accuracy_score(y_train, clf.predict(X_train))
            val_acc = accuracy_score(y_val, clf.predict(X_val))
            test_acc = accuracy_score(y_test, clf.predict(X_test))

            results.append({
                "Split": split_name,
                "Criterion": crit,
                "Train Acc": f"{train_acc:.4f}",
                "Val Acc": f"{val_acc:.4f}",
                "Test Acc": f"{test_acc:.4f}"
            })

    # Print results in a readable format
    results_df = pd.DataFrame(results)
    print(results_df.to_string(index=False))

if __name__ == "__main__":
    main()
```

Q4.8

When we increase the max depth of the tree, the decision tree transitions from a simple model with very few decisions to a complex model that isolates every single data point in the training set. The figure below shows this exactly, as across every data split using Gini impurity, the training accuracy drastically increases as the max depth increases, but the validation and test accuracy falls. This is because overfitting occurs at higher max depths, where the model memorizes the training data, including all its noise and outliers. There are clear trade-offs. With a low max depth, the tree is restricted and cannot split enough, resulting in poor accuracy across training, validation and test sets. With a high max depth, as we saw, the validation and test accuracy decrease but training accuracy increases. The goal is to find a middleground (around 2 or 3 in our case here) where you sacrifice a bit of training accuracy to prevent the tree from memorizing noise and giving poor test and validation performance.

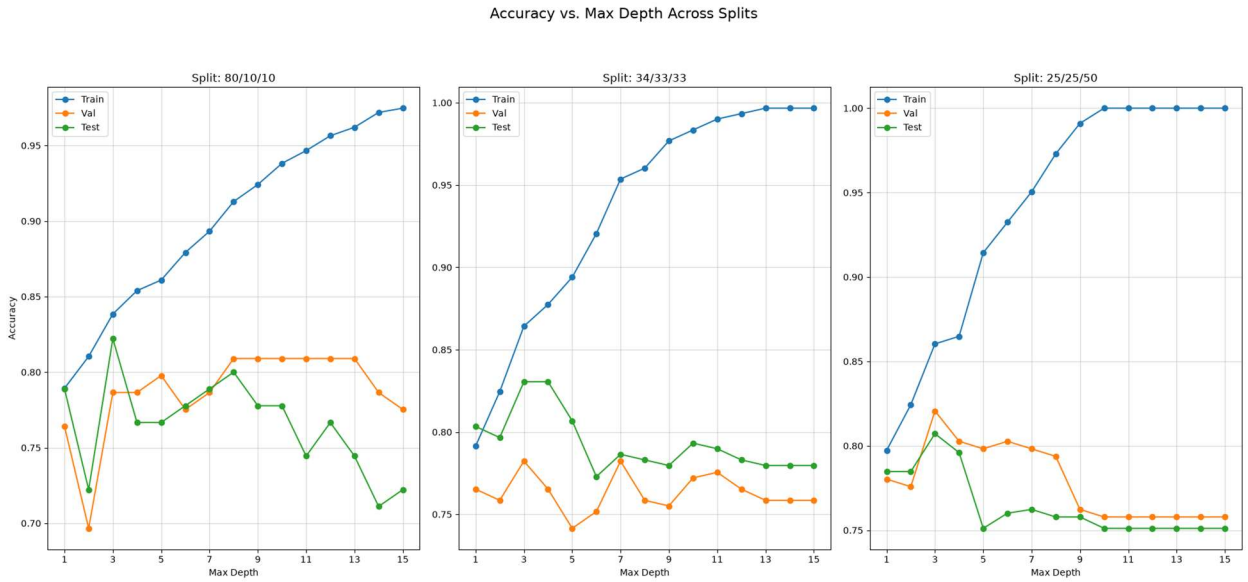


Figure 8: Accuracy vs Max Depth Across Split Data

The code for implementing the graphs is shown below.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load data
df = pd.read_csv("A1Q4-decision-trees.csv")
X = df.drop(columns=['Survived'])
y = df['Survived']

# Define splits
```

```
split_configs = {
    "80/10/10": (0.80, 0.10, 0.10),
    "34/33/33": (0.34, 0.33, 0.33),
    "25/25/50": (0.25, 0.25, 0.50)
}

depths = range(1, 16)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("Accuracy vs. Max Depth Across Splits", fontsize=16)

# Iterate through splits and calculate accuracies
for i, (split_name, (train_r, val_r, test_r)) in
enumerate(split_configs.items()):
    # Split the data
    X_temp, X_test, y_temp, y_test = train_test_split(
        X, y, test_size=test_r, random_state=123123
    )
    val_ratio_adjusted = val_r / (train_r + val_r)
    X_train, X_val, y_train, y_val = train_test_split(
        X_temp, y_temp, test_size=val_ratio_adjusted, random_state=123123
    )

    train_accs, val_accs, test_accs = [], [], []

    # Test different depths
    for depth in depths:
        clf = DecisionTreeClassifier(max_depth=depth, random_state=123123,
criterion='gini')
        clf.fit(X_train, y_train)

        train_accs.append(accuracy_score(y_train, clf.predict(X_train)))
        val_accs.append(accuracy_score(y_val, clf.predict(X_val)))
        test_accs.append(accuracy_score(y_test, clf.predict(X_test)))

    # Plotting
    ax = axes[i]
    ax.plot(depths, train_accs, label='Train', marker='o', linestyle='-')
    ax.plot(depths, val_accs, label='Val', marker='o', linestyle='-')
    ax.plot(depths, test_accs, label='Test', marker='o', linestyle='-')

    ax.set_title(f"Split: {split_name}")
    ax.set_xlabel("Max Depth")
    if i == 0:
        ax.set_ylabel("Accuracy")
    ax.set_xticks(range(1, 16, 2))
```

```
    ax.legend()
    ax.grid(True, alpha=0.5)

plt.tight_layout()
plt.show()
```

GenAI Documentation

Course: ECE 457B

Assignment: Assignment 1

GenAI Platform: Google Gemini

1) Question 3.3

Prompt: " Please generate a function for the following question using sklearn: Q3.3: Compare the two metrics from part I, three variations of train/val/test splits from part II, and additionally with k= 1,3,5,11. Which combination is the best? Explain in terms of train/val/test errors and support with appropriate graphs. {Included all code so far written by me… didn't include because there was so much}"

Response:

```
def question_3_3(X, Y):
    # Ensure splits match your final Q3.2 table
    split_configs = {
        "80/10/10": (0.80, 0.10, 0.10),
        "34/33/33": (0.34, 0.33, 0.33),
        "25/25/50": (0.25, 0.25, 0.50)
    }
    metrics = ["Euclidean", "Cosine"]
    k_values = [1, 3, 5, 11]

    # Set up a 3x2 grid for plotting (3 splits, 2 metrics)
    fig, axes = plt.subplots(3, 2, figsize=(12, 14))
    fig.subplots_adjust(hspace=0.4, wspace=0.3)

    # Store the best configuration found
    best_error = float('inf')
    best_config = {}

    for i, (split_name, (train_r, val_r, test_r)) in
enumerate(split_configs.items()):
        # Split the data using a fixed seed so the graphs are reproducible
        X_temp, X_test, Y_temp, Y_test = train_test_split(
            X, Y, test_size=test_r, random_state=42
        )
        val_ratio_adjusted = val_r / (train_r + val_r)
```

```
        X_train, X_val, Y_train, Y_val = train_test_split(
            X_temp, Y_temp, test_size=val_ratio_adjusted, random_state=42
        )

        for j, metric in enumerate(metrics):
            train_errors = []
            val_errors = []
            test_errors = []

            for k in k_values:
                sk_knn = KNeighborsClassifier(n_neighbors=k,
metric=metric.lower())
                sk_knn.fit(X_train, Y_train)

                # Calculate Error Rates (1 - Accuracy)
                train_err = 1 - accuracy_score(Y_train, sk_knn.predict(X_train))
                val_err = 1 - accuracy_score(Y_val, sk_knn.predict(X_val))
                test_err = 1 - accuracy_score(Y_test, sk_knn.predict(X_test))

                train_errors.append(train_err)
                val_errors.append(val_err)
                test_errors.append(test_err)

                # Track the best performing model based on validation error
                if val_err < best_error:
                    best_error = val_err
                    best_config = {
                        'k': k,
                        'metric': metric,
                        'split': split_name,
                        'train_err': train_err,
                        'val_err': val_err,
                        'test_err': test_err
                    }

            # Plotting the three error lines for this specific split and metric
            ax = axes[i, j]
            ax.plot(k_values, train_errors, label='Train Error', marker='o',
linestyle='-')
            ax.plot(k_values, val_errors, label='Val Error', marker='s',
linestyle='--')
            ax.plot(k_values, test_errors, label='Test Error', marker='^',
linestyle=':')

            ax.set_title(f"Split: {split_name} | Metric: {metric}")
```

```
            ax.set_xlabel("Number of Neighbors (k)")
            ax.set_ylabel("Error Rate")
            ax.set_xticks(k_values)
            ax.legend()
            ax.grid(True, alpha=0.5)

    plt.suptitle("KNN Error Rates Across Splits, Metrics, and k-values",
fontsize=16)
    plt.show()

    # FIXED: Restored to .4f precision so you can actually read the exact results
    print(f"\n{'='*40}")
    print("=== BEST COMBINATION IDENTIFIED ===")
    print(f"{'='*40}")
    print(f"Split:            {best_config['split']}")
    print(f"Metric:           {best_config['metric']}")
    print(f"k-value:          {best_config['k']}")
    print(f"Training Error:   {best_config['train_err']:.4f}")
    print(f"Validation Error: {best_config['val_err']:.4f}")
    print(f"Test Error:       {best_config['test_err']:.4f}")
```